

OSLAB PRACTICAL-12 PROGRAMS & OUTPUT

Date: 03-10-26

Program-1 CODE & OUTPUT:

```
sainath@sainath:~/OSLAB/practical_12$ cat program1.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 1; i <= 10; i++) {

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= 10; i++) {

        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }

    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

```
sainath@Sainath:~/OSLAB/practical_12$ ./program1
```

```
Produced: 1
```

```
Produced: 2
```

```
Produced: 3
```

```
Produced: 4
```

```
Produced: 5
```

```
Consumed: 1
```

```
Consumed: 2
```

```
Consumed: 3
```

```
Consumed: 4
```

```
Consumed: 5
```

```
Produced: 6
```

```
Produced: 7
```

```
Produced: 8
```

```
Produced: 9
```

```
Produced: 10
```

```
Consumed: 6
```

```
Consumed: 7
```

```
Consumed: 8
```

```
Consumed: 9
```

```
Consumed: 10
```

Program-2 CODE & OUTPUT:

```
sainath@Sainath:~/OSLAB/practical_12$ gedit program2.c
sainath@Sainath:~/OSLAB/practical_12$ gcc program2.c -o program2
sainath@Sainath:~/OSLAB/practical_12$ cat program2.c

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

void *thread1(void *arg) {
    pthread_mutex_lock(&lock1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&lock2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void *thread2(void *arg) {
    pthread_mutex_lock(&lock2);
    printf("Thread 2 locked Resource 2\n");

    sleep(1);

    pthread_mutex_lock(&lock1);
    printf("Thread 2 locked Resource 1\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

```
sainath@Sainath:~/OSLAB/practical_12$ ./program2
Thread 2 locked Resource 2
Thread 1 locked Resource 1
^C
```

Program-3 CODE & OUTPUT:

```
sainath@sainath:~/OSLAB/practical_12$ gedit program3.c
sainath@sainath:~/OSLAB/practical_12$ gcc program3.c -o program3
sainath@sainath:~/OSLAB/practical_12$ cat program3.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

void *thread1(void *arg) {
    pthread_mutex_lock(&lock1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&lock2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void *thread2(void *arg) {
    pthread_mutex_lock(&lock1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&lock2);
    printf("Thread 2 locked Resource 2\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    printf("Program completed without deadlock.\n");

    return 0;
}
```

```
sainath@sainath:~/OSLAB/practical_12$ ./program3
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Program completed without deadlock.
```

History:

```
609 cd OSLAB
610 ls
611 mkdir practical_12
612 cd practical_12
613 gedit program1.c
614 gcc program1.c -o program1
615 ./program1
616 cat program1.c
617 ./program1
618 gedit program2.c
619 gcc program2.c -o program2
620 cat program2.c
621 ./program2
622 gedit program3.c
623 gcc program3.c -o program3
624 cat program3.c
625 ./program3
626 history
```